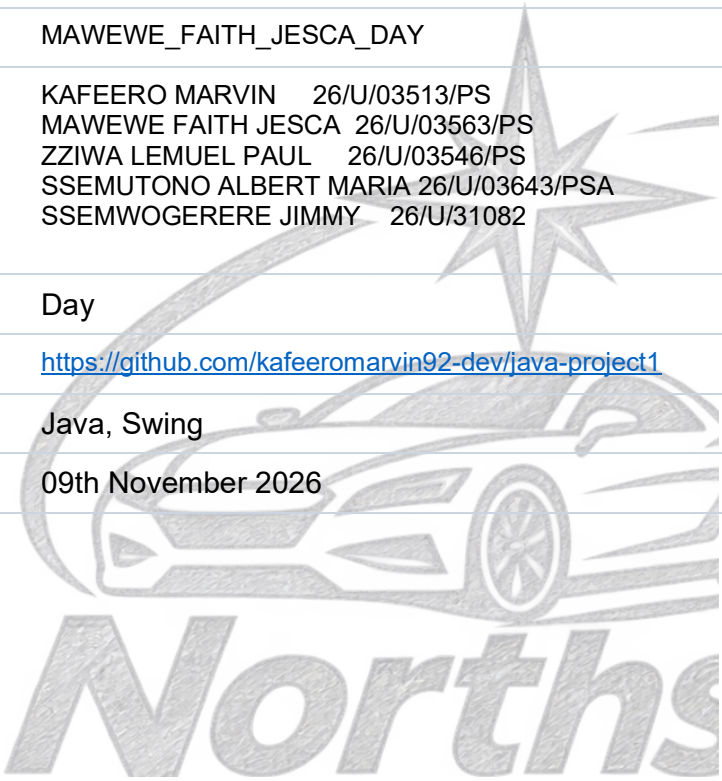
CSC 1102: Structured and Object-Oriented Programming

Practical Exam Project

Vehicle Rental System

Project Documentation

Group leader:	MAWEWE_FAITH_JESCA_DAY
Members:	KAFEERO MARVIN 26/U/03513/PS MAWEWE FAITH JESCA 26/U/03563/PS ZZIWA LEMUEL PAUL 26/U/03546/PS SSEMUTONO ALBERT MARIA 26/U/03643/PSA SSEMWOGERERE JIMMY 26/U/31082
Session:	Day
GitHub link:	https://github.com/kafeeromarvin92-dev/java-project1
Language / GUI:	Java, Swing
Submission date:	09th November 2026



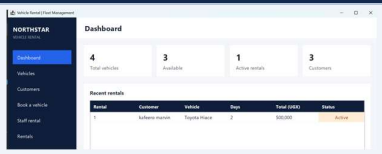
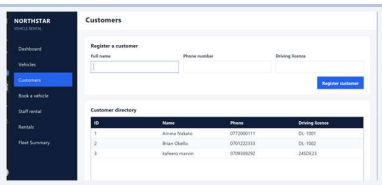
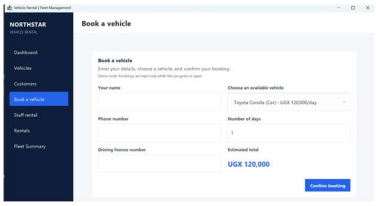
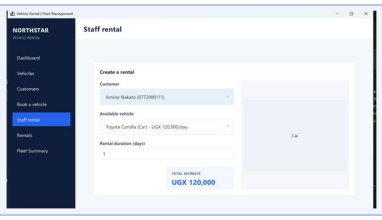
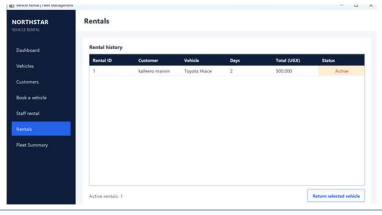
1. Description of the Application

The Vehicle Rental System is our project written in Java with a Swing and graphical interface. It helps small rental company keep record of its fleet (cars, vans, motorcycles and luxury cars), its customers and its rentals. Staff members can add vehicles with photos, register its customers, rent a vehicle out, record its return and view a summary of the whole fleet on a dashboard. A customer can also register themselves and book an available vehicle directly from “Book a vehicle” class, and is given a booking receipt with the total cost of everything combined.

The program stores all of its data in arrays while it is running because there is no database used as instructions by the project examination were. Some sample vehicles and customers are loaded at start-up so the program can be demonstrated immediately on our dashboard. Prices are shown in Uganda shillings (UGX).

2. MAIN FEATURES

Screenshots of the system pages and a brief explanation of each feature

Page	Screenshot	What it does
Dashboard		Displays the total vehicles, available vehicles, active rentals and customers. It also shows the latest rental records and their status.
Vehicles		Allows staff to add vehicles by entering the make, model, type and daily rate, with an optional photo and luxury-car fee.
Customers		Allows staff to register customers using their name, phone number and driving-licence details, and view the customer directory.
Book a vehicle		Allows a customer to enter their details, choose an available vehicle and number of days, view the estimated cost and confirm a booking.
Staff rental		Allows staff to select an existing customer and an available vehicle, enter the rental period, review the cost and confirm the rental.
Rentals		Displays rental records, including the customer, vehicle, number of days, total cost and status. Staff can confirm a vehicle return.

Pricing rules

The total cost is calculated according to the number of days rented and price of the car per day.

3. Classes and Their Purpose

The project has nine classes in six files as listed below.

Class (file)	Purpose	Important members
Vehicle (Vehicle.java)	Parent class for every vehicle. Holds the vehicle's data and its availability.	Private fields id, make, model, type, dailyRate, available, imagePath; constructor; getters; setters (setAvailable, setDailyRate checks rate > 0, setImagePath); calculateCost(days); getDetails(); toString().
LuxuryCar* (Vehicle.java)	Child of Vehicle: a vehicle with an extra premium service fee.	Extra field luxuryServiceFee; constructor calls super(...); overrides calculateCost() and getDetails().
Customer (Customer.java)	Stores all the details of one customer.	Private fields id, name, phone, licenseNumber; constructor; getters; setName, setPhone; toString() for dropdown lists.
Rental (Rental.java)	Stores one booking between a customer and a vehicle and works out its price.	Fields id, vehicle, customer, days, totalCost, active; constructor marks the vehicle as taken; computeCost() with the discount switch; returnVehicle().
RentalManager (RentalManager.java)	The "brain" of the program. Owns the arrays and performs all the operations.	Vehicle[50], Customer[50], Rental[100] with counters; addVehicle, getAvailableVehicles, addCustomer, rentVehicle, returnRental, getActiveRentalCount, addSampleData.
VehicleAlreadyRentedException* (RentalManager.java)	Custom exception for trying to rent a vehicle that is already rented.	Extends Exception; constructor passes the message to super(message).
MainFrame (MainFrame.java)	The Swing window: sidebar, seven pages, forms, tables, receipts and error messages.	Extends JFrame; one shared RentalManager; CardLayout to switch pages; methods such as addVehicle, registerCustomer, createCustomerBooking, rentVehicle, returnVehicle, refreshAll.
ImageHelper* (MainFrame.java)	Loads vehicle photos from the images folder, crops them to fit, or draws a placeholder if the file is missing.	loadIcon(), findFile(), fitImage(), createPlaceholder().
Main (Main.java)	Starting point of the program. Contains the group details as comments.	main() sets the look and feel and opens MainFrame using SwingUtilities.invokeLater.

4. How the Classes Interact

The program follows a simple separation: **MainFrame** handles what the user sees, **RentalManager** handles the data and rules, and **Vehicle, LuxuryCar, Customer and Rental** are the data objects stored by the manager.

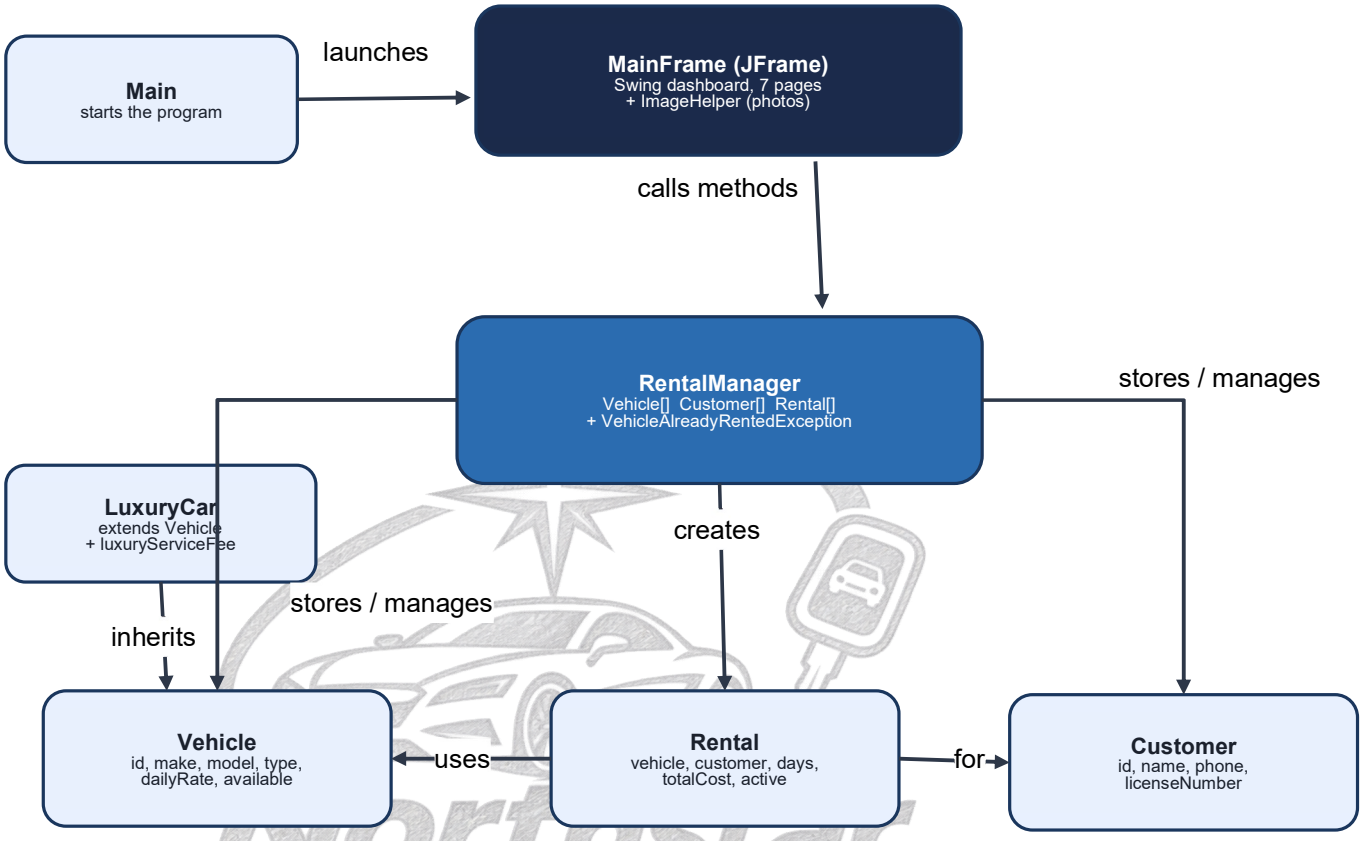


Figure 1: Class relationships. Solid arrows show "uses / creates"; the hollow arrow shows inheritance.

- **Start-up:** Main sets the look and feel and creates a MainFrame. MainFrame's constructor creates one RentalManager, calls addSampleData() and builds the pages.
- **Fleet storage and polymorphism:** addVehicle() creates either a Vehicle or a LuxuryCar and stores it in the same Vehicle[] array. When the Fleet Summary or a price is needed, the program calls getDetails() or calculateCost() on each Vehicle and Java runs the correct version (the LuxuryCar one adds the service fee).
- **Renting a vehicle:** MainFrame reads the form, then calls RentalManager.rentVehicle(vehicle, customer, days). The manager validates the request, creates a Rental, and the Rental constructor marks the Vehicle as not available and calculates the cost. If the vehicle is already taken, a VehicleAlreadyRentedException is thrown and MainFrame shows the message in a warning dialog.
- **Returning a vehicle:** MainFrame calls returnRental(index); the Rental becomes inactive and its Vehicle is set available again, so it appears again in the booking dropdowns.
- **Refreshing the screens:** after every change MainFrame calls refreshAll(), which re-reads the arrays through RentalManager's getters and updates the vehicle cards, tables, dropdowns, dashboard totals and fleet summary.
- **Photos:** MainFrame asks ImageHelper for an image for each vehicle; ImageHelper returns the photo or a placeholder, so the program never crashes on a missing file.

5. Programming Concepts Demonstrated

Requirement	Where it is used
Classes, objects, constructors	➢ Vehicle, LuxuryCar, Customer, Rental, RentalManager; objects created with *new*, e.g. new Rental(...) inside rentVehicle().
Access modifiers, getters and setters	➢ All fields are private; public getters/setters such as getDailyRate(), setAvailable(), setPhone(). setDailyRate() only accepts values above zero.
Inheritance	➢ LuxuryCar extends Vehicle and uses super(...) in its constructor.
Polymorphism / overriding	➢ LuxuryCar overrides calculateCost() and getDetails(); the Vehicle[] array holds both kinds.
Custom exception	➢ VehicleAlreadyRentedException, thrown in rentVehicle() and caught in MainFrame.
Arrays of objects	➢ Vehicle[] vehicles, Customer[] customers, Rental[] rentals in RentalManager; getAvailableVehicles() builds a new Vehicle[].
if / else	➢ Validation of every form and rule, e.g. checking for empty fields, duplicate licence numbers, vehicle availability.
Switch	➢ Rental.computeCost() chooses the discount (0, 1 or more weeks).
for loop	➢ Searching and filling arrays, e.g. getAvailableVehicles(), refreshCustomerTable().
while loop	➢ RentalManager.getActiveRentalCount() counts active rentals.
do-while loop	➢ RentalManager.nextVehicleId() keeps trying until it finds an unused vehicle number.
try / catch	➢ NumberFormatException when reading numbers from text fields; VehicleAlreadyRentedException when renting; IOException when loading photos.
Swing GUI	➢ JFrame, JPanel, CardLayout, JTable, JComboBox, JTextField, JButton, JOptionPane dialogs, JFileChooser for photos.

6. How to Run the Program

- Keep all .java files together in one folder, with the “images” folder beside them.
- In a terminal run javac *.java and then java Main, or open the folder in an IDE and run Main.java.
- To add a vehicle photo, copy a .jpg or .png into the images folder and choose it on the Vehicles page. If a photo is missing, a placeholder is shown.
- ;c3eThis is a classroom demo on one computer: all data is cleared when the program closes.\